

网络基础

HTTP 与 HTTPS

HTTP（超文本传输协议）是 Web 应用最基础的协议，它运行在 TCP

之上，采用请求-响应模型：客户端（浏览器、App）发起请求，服务器返回响应。HTTP 的请求包含请求方法、URL、请求头和请求体，响应包含状态码、响应头和响应体。常见的请求方法有 GET（获取资源）、POST（创建资源）、PUT（整体更新）、PATCH（部分更新）、DELETE（删除资源），此外还有 HEAD、OPTIONS 等。状态码方面，2xx 表示成功，3xx 表示重定向，4xx 表示客户端错误，5xx 表示服务器错误。

HTTP 最大的问题是明文传输。在网络传输的每一个环节——局域网、路由器、运营商——数据都裸露在数据包里，很容易被窃听和篡改。比如在公共 WiFi 上访问纯 HTTP 网站，攻击者可以轻松截获你的用户名密码。此外 HTTP 也不验证通信对方的身份，中间人完全可以冒充服务器。正是这些安全问题催生了 HTTPS。

HTTPS 在 HTTP 和 TCP 之间加了一层 SSL/TLS 加密。它主要解决三个问题：机密性，数据经过对称加密，中间人无法读取内容；完整性，通过消息验证码校验数据没有被篡改；身份认证，通过数字证书验证服务器的真实身份。TLS 握手的大致过程是：客户端发送支持的加密套件，服务器返回证书和公钥，客户端验证证书后生成会话密钥并用服务器公钥加密发送，双方用这个会话密钥进行后续的对称加密通信。证书由受信任的 CA（证书颁发机构）签发，浏览器内置了 CA 根证书，所以能验证服务器身份。

现代 Web 已经全面进入 HTTPS 时代：搜索引擎优先收录 HTTPS 站点，浏览器对纯 HTTP 站点会打上「不安全」的标记。对于需要登录的网站，HTTPS 更是基本要求，否则账号密码等于裸奔。配置 HTTPS 现在也很简单，Let's Encrypt 等 CA 提供免费证书，配合 Nginx 几分钟就能部署。

TCP 三次握手

TCP 是面向连接的可靠传输协议，通信双方在传输数据前必须先建立连接，这个建立连接的过程就是著名的三次握手。为什么叫三次握手？因为需要交换三组报文：第一次，客户端向服务器发送 SYN 报文，并携带自己的初始序列号，表示「我想和你建立连接」；第二次，服务器收到后回复 SYN+ACK 报文，确认收到客户端的 SYN，同时带上自己的初始序列号，表示「我收到你的请求了，我也准备好了」；第三次，客户端再发送 ACK 报文确认服务器的 SYN，双方随即进入已连接状态。

为什么一定是三次而不是两次？核心原因是为了确认双方的收发能力都正常，并协商好初始序列号。只有客户端发了、服务器收到并回应、客户端再确认服务器能收到，双方才能确定「我的数据能发到你那，你的数据也能发到我这里」。如果只有两次握手，服务器无法确认客户端是否真的收到了自己的报文；同时，两次握手无法处理「迟到的重复连接请求」问题：一个迟到的旧 SYN 到达服务器，如果服务器直接建立连接，就会白白浪费资源。三次握手时，客户端收到迟到的确认会直接丢弃或回复 RST，连接不会误建立。

与建立连接对应的是四次挥手，用于关闭连接。为什么断开要四次？因为 TCP 是全双工的，两个方向的关闭是独立的：一方发送 FIN 表示「我不再发数据了」，对方回复 ACK 确认，然后对方也发送 FIN，这一方再回复 ACK，连接才彻底关闭。过程中还可能存在半关闭状态，即一方已关闭发送、另一方仍在发送。另外，主动关闭方最后要进入 TIME_WAIT 状态等待 2MSL 时间，确保最后一个 ACK 送达，防止新连接收到旧连接的迟到报文。

TCP 的可靠性还体现在其他机制上：滑动窗口实现流量控制，拥塞控制（慢启动、拥塞避免、快速重传）调节发送速率避免网络过载，序列号 + 确认重传保证数据不丢不乱。这些机制让 TCP 成为可靠传输的基石，但同时也有一定的头部开销和延迟，因此实时音视频、游戏这类对延迟敏感的场景会改用 UDP。

UDP 与 TCP 的对比

UDP（用户数据报协议）是无连接的、不可靠的传输协议：发送方只管把数据报扔到网络上，不建立连接、不确认、不重传、不保证顺序。它没有 TCP 的握手和拥塞控制开销，延迟低、实时性好。适用场景包括：实时音视频通话（允许少量丢包）、在线游戏（延迟优先）、DNS 查询、DHCP。TCP

适合对可靠性要求高的场景：网页浏览、文件传输、邮件、数据库连接。简单说，TCP 用「慢一点」换「不丢不乱」，UDP 用「可能丢」换「快」。很多应用会结合两者：比如视频通话的媒体流走 UDP，信令控制走 TCP。

Cookie 与 Session

HTTP 是无状态协议，服务器不记得上一次请求是谁发的。为了让网站记住用户（登录状态、购物车），引入了 Cookie 和 Session 机制。Cookie 是保存在客户端浏览器的一小段数据，由服务器通过 Set-Cookie 响应头下发，浏览器后续每次请求都会自动带上。Session 是保存在服务器端的数据，客户端只保存一个 session id，服务器用这个 id 在内存或 Redis 中找到对应的会话数据。

Cookie 和 Session 各有优缺点。Cookie 存在客户端，容量小（一般 4KB 左右）、可以被用户查看和修改、存在被 XSS 窃取的风险，但服务器无状态、好扩展；Session 存在服务端，容量大、更安全、可以主动失效，但服务器有状态，多台机器部署时必须共享 Session（通常存 Redis）。两者经常配合使用：登录成功后服务器创建 Session，把 session id 写进 Cookie，之后每次请求浏览器带 Cookie，服务器查 Session 判断登录态。针对 Cookie 的安全问题，要注意几个要点：HttpOnly 属性让 JavaScript 无法读取 Cookie，能有效防止 XSS 窃取；Secure 属性让 Cookie 只在 HTTPS 连接下传输；SameSite 属性可以限制跨站携带 Cookie，防止 CSRF 攻击。除此之外，现代应用越来越多使用 Token 方案（如

JWT)：把用户信息编码进签名后的 token 里，服务器验签即可，无需存储，天然适合分布式和无状态场景。登录态方案的选择取决于场景：传统 Web 用 Session，前后端分离、移动端接口用 JWT 更常见。

DNS 与域名解析

当我们访问 `www.example.com` 时，浏览器首先需要知道这个域名对应的服务器 IP 地址，这个查询过程就是 DNS 解析。DNS 是一个分层的分布式系统：本地缓存 → 递归服务器 → 根服务器 → 顶级域服务器 → 权威服务器，逐级找到 IP。整个过程通常在几十毫秒内完成，大部分查询都有缓存。常见的 DNS 记录类型有 A (IPv4 地址)、AAAA (IPv6 地址)、CNAME (别名)、MX (邮件服务器)、TXT (文本记录，常用于域名验证)。DNS 是互联网的基础设施，一旦 DNS 故障，整个网站都会无法访问。

IP 地址与端口

IP 地址用于定位网络中的一台设备：IPv4 是 32 位，如 `192.168.1.1`；IPv6 是 128 位，解决 IPv4 地址枯竭问题。端口号用于区分一台设备上的不同服务：一个 IP 地址加上一个端口就能唯一定位一个服务。常见端口：HTTP 是 80，HTTPS 是 443，SSH 是 22，MySQL 是 3306，Redis 是 6379。公网 IP 有限，内网设备通过 NAT 共享公网出口，所以从公网访问内网服务需要端口映射或内网穿透。

TCP/IP 分层模型

理解网络分层是排查网络问题的基础。TCP/IP 模型分为四层：应用层 (HTTP、DNS、FTP)、传输层 (TCP、UDP)、网络层 (IP、ICMP)、网络接口层 (以太网、WiFi)。OSI 七层模型把它进一步细分，但实际通信主要遵循 TCP/IP 四层。分层的好处是每层只关心自己的职责，下层为上层提供服务。排查问题时也按层来：先看应用 (错误码)、再看端口和传输 (连通性)、再看网络 (IP、路由)、最后看物理链路。